Reflection homework 14

Do Huy Du - 202417108

May 31, 2026

1 Main functionalities

This homework rewrites the Date Display GUI from Swing to JavaFX. On startup the app shows today's date in the Vietnamese format and lets the user change four things: the display format (Vietnamese, US, or European, through a `ComboBox`), the base date (through a `DatePicker`), the shift amount x, and the operation itself — Before or After by Days, Months, or Years. The selected date stays on top while the calculated result lives in its own labelled box below the Apply button, so the original choice is never lost. Validation errors appear in red as the user types, and the Apply button disables itself whenever the form is invalid.

2 Design decisions — how MVC was applied

`DateModel` only knows about state. The selected date, the result date, and the active format are all `ObjectProperty` fields, and the model does not import anything from the controller package anymore — a circular dependency I had carried over from hw13b. `DateController` is a thin glue layer: it wires the FXML widgets to model properties inside `initialize()` and exposes a single `applyOperation()` method. The View is pure FXML with inline styling, so the controller never reaches into UI internals and the model never touches a JavaFX node.

3 Improvements over the previous version

The Swing version mutated one shared `LocalDate` on every button press, which meant the selected date and the result were literally the same object — after Apply, the original choice was gone. This time they are two separate `ObjectProperty<LocalDate>` fields, and the displayed text is always in sync with the chosen format because both the selected-date label and the result label are bound to a `StringBinding` that depends on the date *and* the format together. Validation is reactive instead of being checked only after a click, and the Apply button reflects that state without needing any manual `setDisable` calls.

4 JavaFX properties, binding, and observable collections

The core requirement of this homework lives in `Bindings.createStringBinding`: the displayed-date label recomputes automatically whenever either the selected date or the format property changes, which is much cleaner than the `refreshDisplay()` pattern I used in hw13b. The Apply button uses `disableProperty().bind(errorBinding.isNotEmpty())` to gate user input, and the format `ComboBox` is populated with an `ObservableList` built from the `DateFormat` enum values so its selection can also be bound bidirectionally to the model.